

Exercices DART

Sommaire

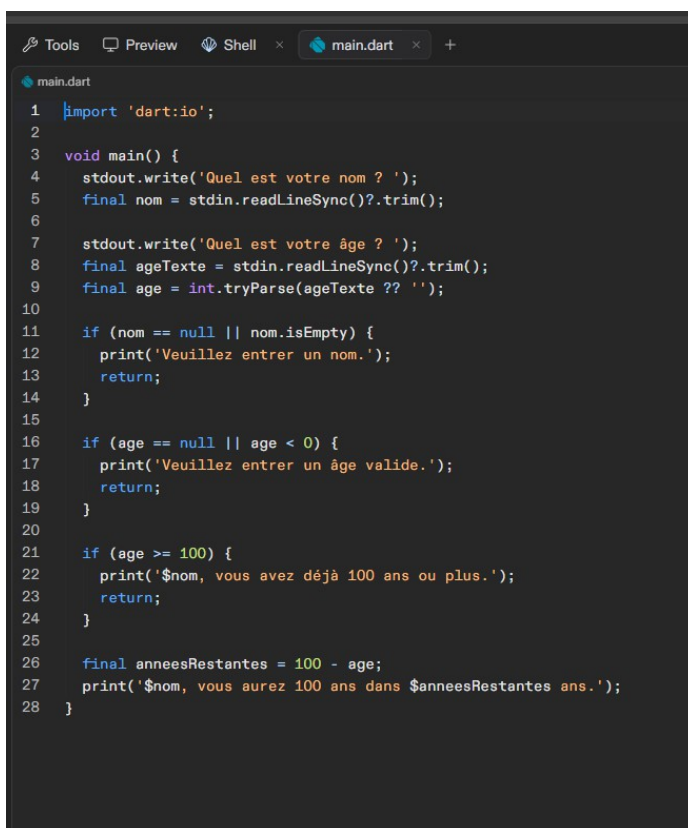
<u>Introduction</u>	2
<u>Exercice :1</u>	2
<u>Exercice :2</u>	4
<u>Exercice :3</u>	5
<u>Exercice :4</u>	6
<u>Exercice :5</u>	6
<u>Exercice :6</u>	6
<u>Exercice :7</u>	6
<u>Exercice :8</u>	6
<u>Exercice :9</u>	6
<u>Exercice :10</u>	6
<u>Exercice :11</u>	6
<u>Exercice :12</u>	6
<u>Exercice :13</u>	6
<u>Exercice :14</u>	6
<u>Exercice :15</u>	6
<u>Exercice :16</u>	6
<u>Exercice :17</u>	6
<u>Exercice :18</u>	6
<u>Exercice :19</u>	6
<u>Exercice :20</u>	6
<u>Exercice :21</u>	6
<u>Exercice :22</u>	6
<u>Exercice :23</u>	7
<u>Exercice :24</u>	7
<u>Exercice :25</u>	7
<u>Exercice :26</u>	7
<u>Conclusion</u>	7

Introduction

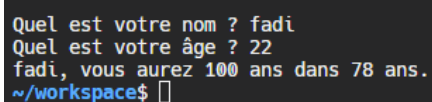
Dans ce TP, nous avons réalisé plusieurs exercices en langage Dart afin de découvrir et de pratiquer les bases de la programmation. Nous avons travaillé avec les variables, les conditions, les boucles, les fonctions et les listes.

Exercice :1

Créez un programme qui demande à l'utilisateur d'entrer son nom et son âge. Affichez un message qui indique dans combien d'années vous aurez 100 ans.

A screenshot of an IDE window titled 'main.dart'. The code is as follows:

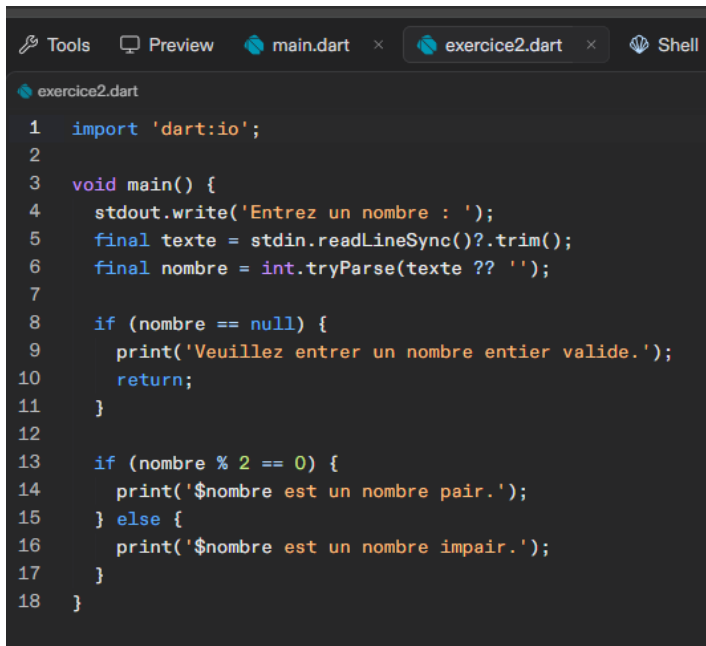
```
1 import 'dart:io';
2
3 void main() {
4   stdout.write('Quel est votre nom ? ');
5   final nom = stdin.readLineSync()?.trim();
6
7   stdout.write('Quel est votre âge ? ');
8   final ageTexte = stdin.readLineSync()?.trim();
9   final age = int.tryParse(ageTexte ?? '');
10
11   if (nom == null || nom.isEmpty) {
12     print('Veuillez entrer un nom.');
```

A screenshot of a terminal window showing the execution of the Dart program. The output is:

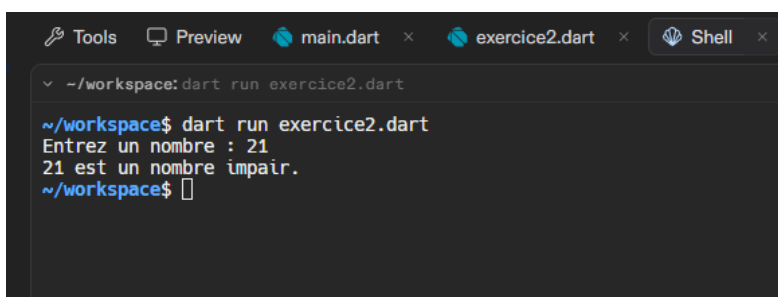
```
Quel est votre nom ? fadi
Quel est votre âge ? 22
fadi, vous aurez 100 ans dans 78 ans.
~/workspace$
```

Exercice :2

Demandez un numéro à l'utilisateur. Selon que le nombre est pair ou impair, imprimez un message approprié à l'utilisateur.



```
exercice2.dart
1  import 'dart:io';
2
3  void main() {
4      stdout.write('Entrez un nombre : ');
5      final texte = stdin.readLineSync()?.trim();
6      final nombre = int.tryParse(texte ?? '');
7
8      if (nombre == null) {
9          print('Veuillez entrer un nombre entier valide.');
```



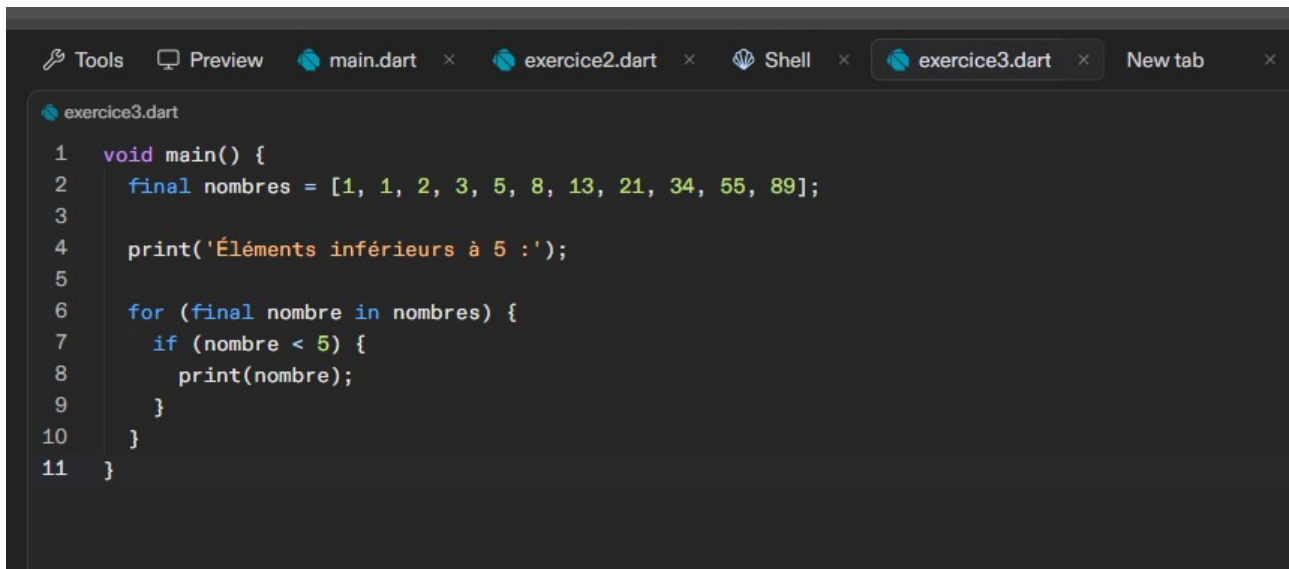
```
~/workspace:dart run exercice2.dart
~/workspace$ dart run exercice2.dart
Entrez un nombre : 21
21 est un nombre impair.
~/workspace$
```

Exercice :3

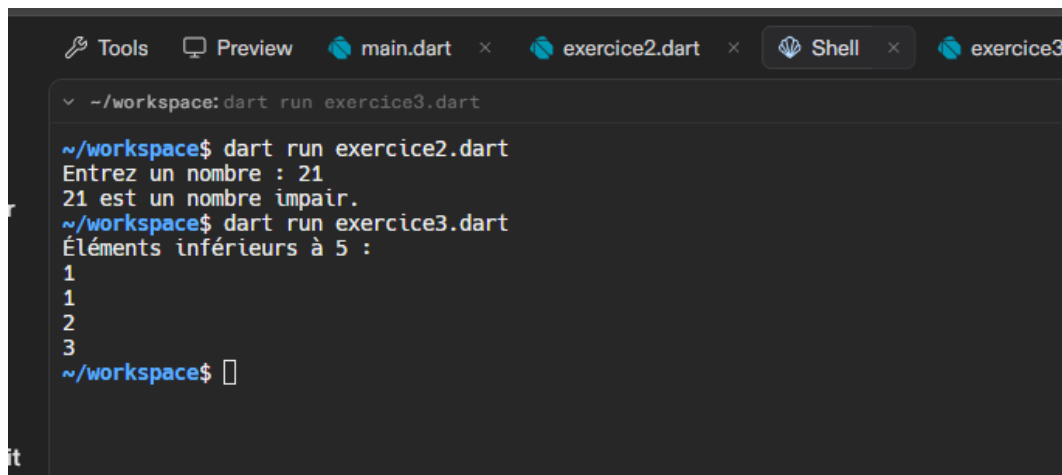
Prenez une liste, dites par exemple celle-ci :

$a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]$

et écrivez un programme qui imprime tous les éléments de la liste qui sont inférieurs à 5.



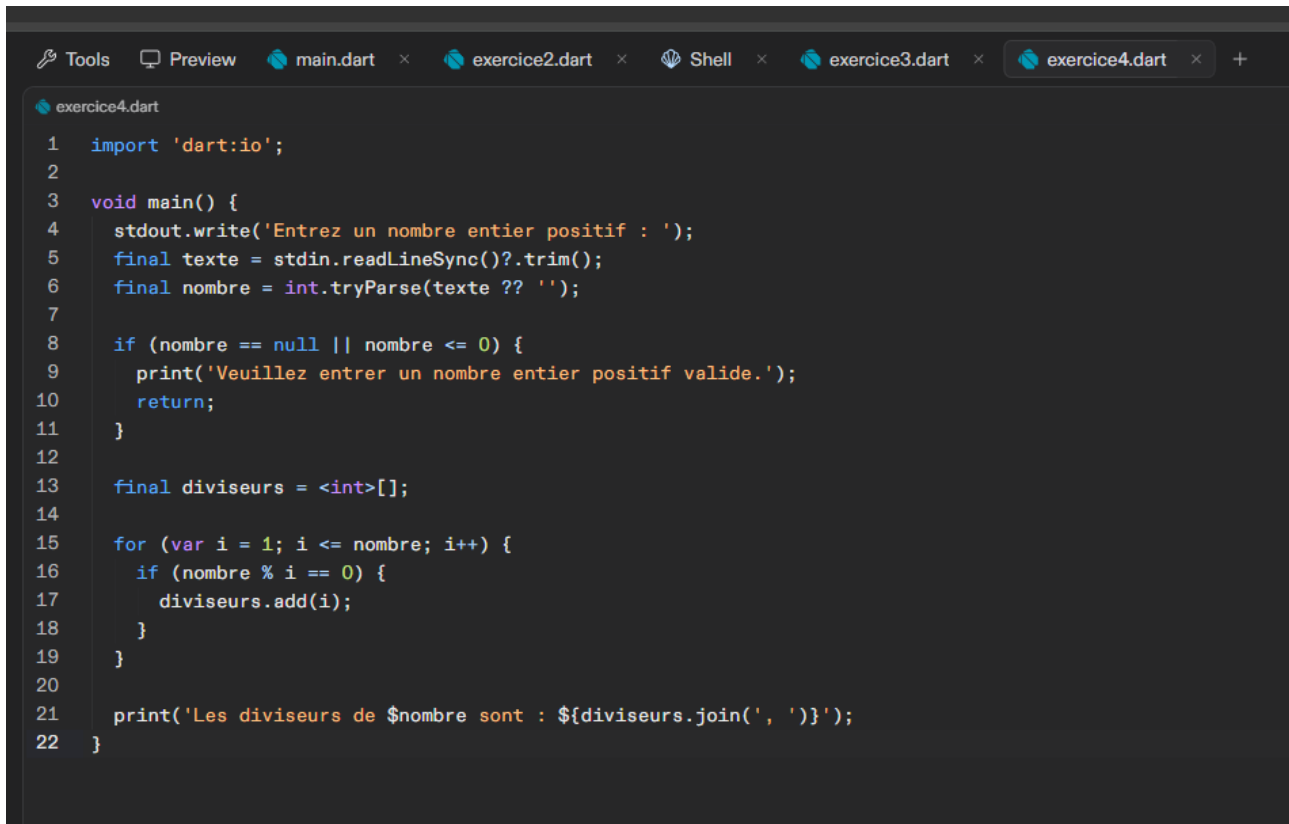
```
exercice3.dart
1 void main() {
2   final nombres = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];
3
4   print('Éléments inférieurs à 5 :');
5
6   for (final nombre in nombres) {
7     if (nombre < 5) {
8       print(nombre);
9     }
10  }
11 }
```



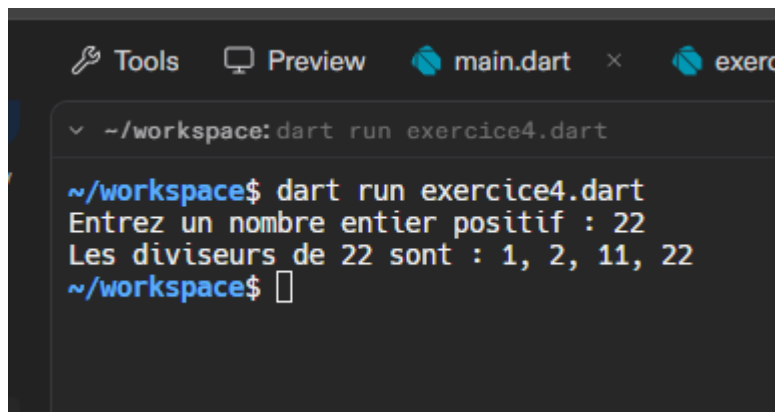
```
~/workspace: dart run exercice3.dart
~/workspace$ dart run exercice2.dart
Entrez un nombre : 21
21 est un nombre impair.
~/workspace$ dart run exercice3.dart
Éléments inférieurs à 5 :
1
1
2
3
~/workspace$
```

Exercice :4

Créez un programme qui demande à l'utilisateur un nombre, puis imprime une liste de tous les diviseurs de ce nombre.

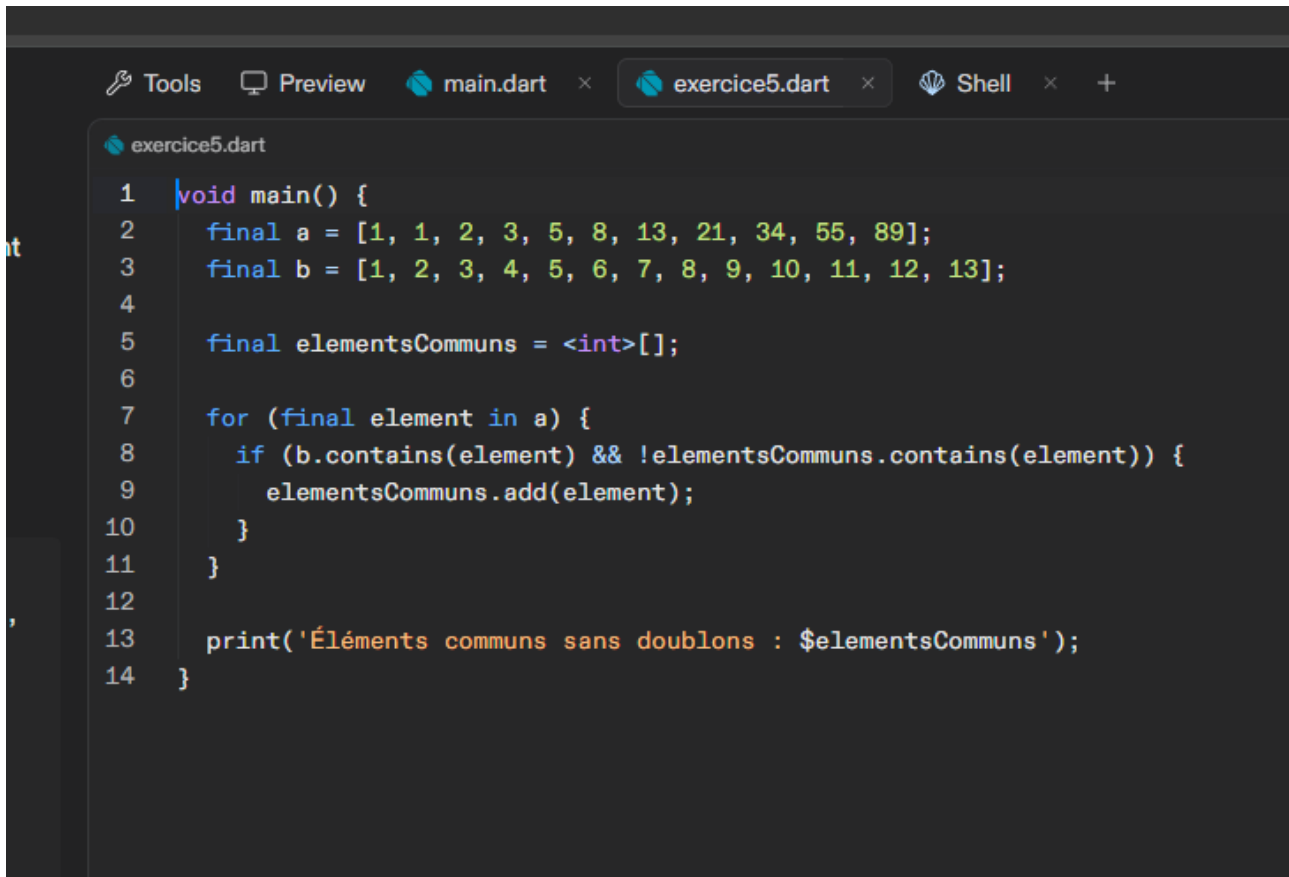


```
exercice4.dart
1  import 'dart:io';
2
3  void main() {
4      stdout.write('Entrez un nombre entier positif : ');
5      final texte = stdin.readLineSync()?.trim();
6      final nombre = int.tryParse(texte ?? '');
7
8      if (nombre == null || nombre <= 0) {
9          print('Veuillez entrer un nombre entier positif valide.');
```

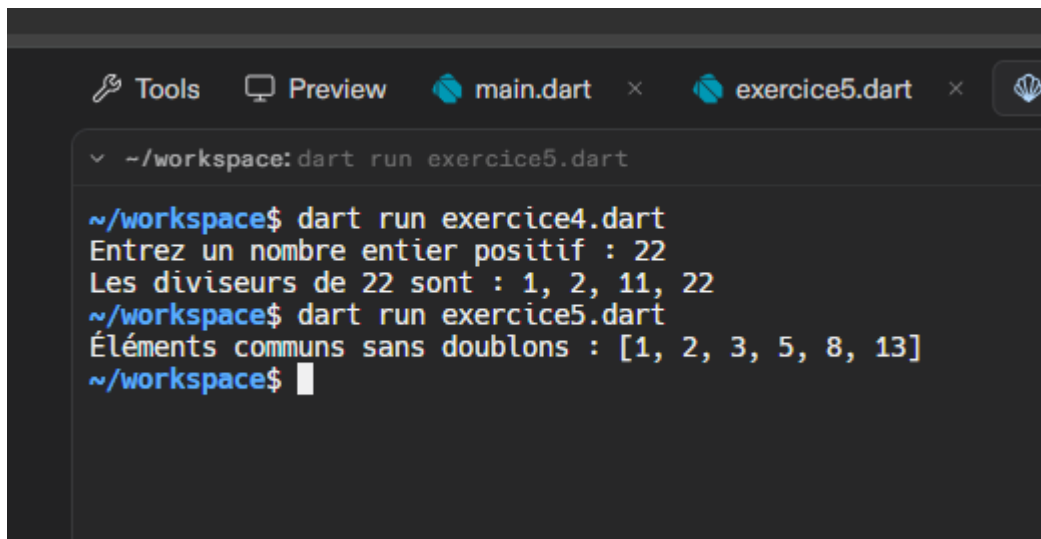


```
~/workspace:dart run exercice4.dart
~/workspace$ dart run exercice4.dart
Entrez un nombre entier positif : 22
Les diviseurs de 22 sont : 1, 2, 11, 22
~/workspace$
```

Exercice :5



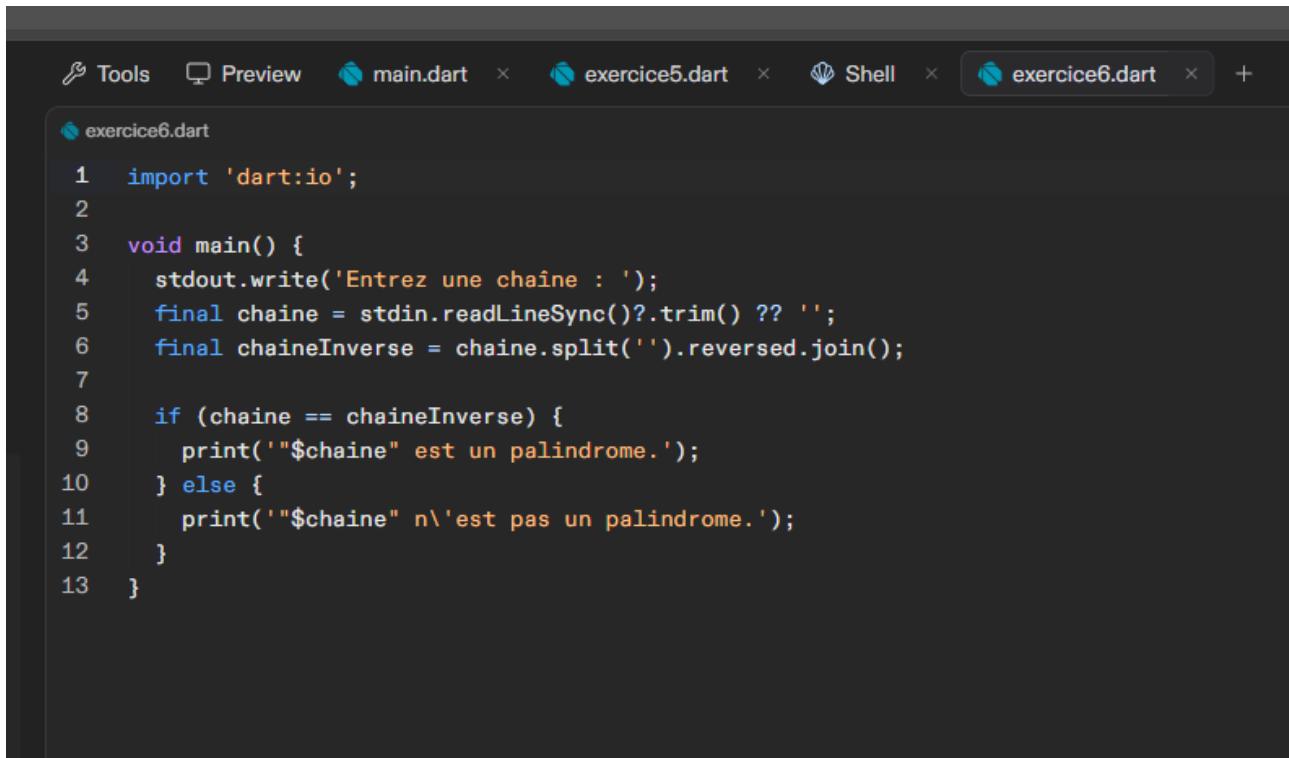
```
1 void main() {
2   final a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];
3   final b = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13];
4
5   final elementsCommuns = <int>[];
6
7   for (final element in a) {
8     if (b.contains(element) && !elementsCommuns.contains(element)) {
9       elementsCommuns.add(element);
10    }
11  }
12
13  print('Éléments communs sans doublons : $elementsCommuns');
14 }
```



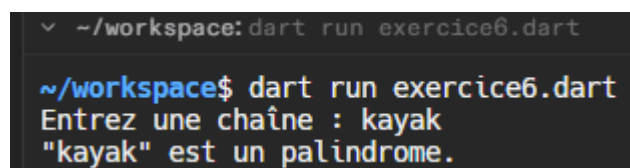
```
~/workspace$ dart run exercice4.dart
Entrez un nombre entier positif : 22
Les diviseurs de 22 sont : 1, 2, 11, 22
~/workspace$ dart run exercice5.dart
Éléments communs sans doublons : [1, 2, 3, 5, 8, 13]
~/workspace$
```

Exercice :6

Demandez à l'utilisateur une chaîne et imprimez si cette chaîne est un palindrome ou non.



```
Tools Preview main.dart x exercice5.dart x Shell x exercice6.dart x +
exercice6.dart
1 import 'dart:io';
2
3 void main() {
4   stdout.write('Entrez une chaîne : ');
5   final chaine = stdin.readLineSync()?.trim() ?? '';
6   final chaineInverse = chaine.split('').reversed.join();
7
8   if (chaine == chaineInverse) {
9     print('$chaine est un palindrome.');
```



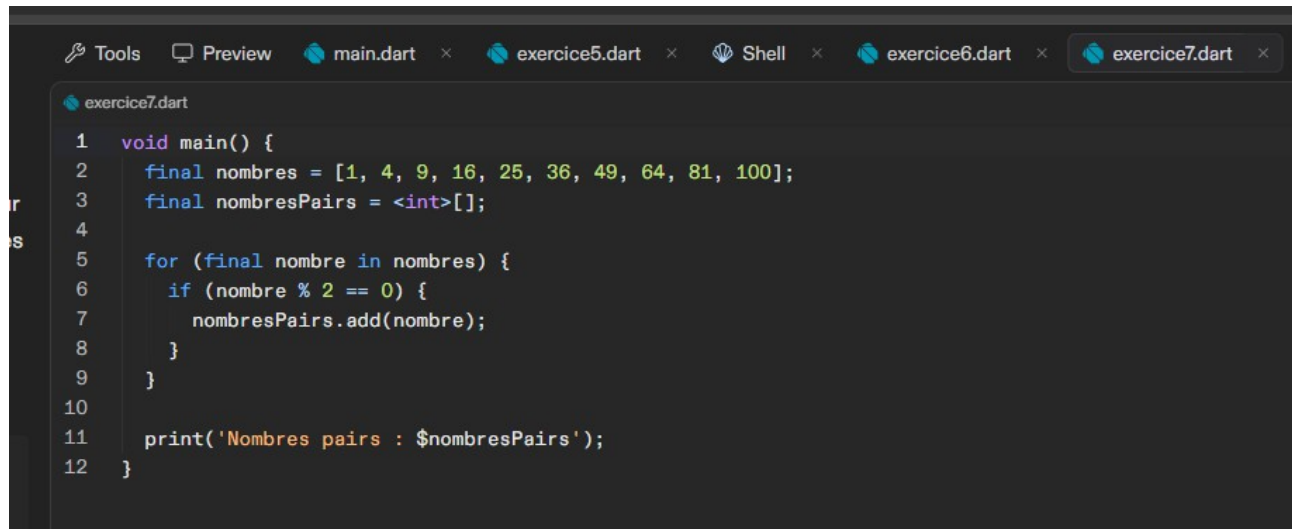
```
~/workspace: dart run exercice6.dart
~/workspace$ dart run exercice6.dart
Entrez une chaîne : kayak
"kayak" est un palindrome.
```

Exercice :7

Disons qu'on vous donne une liste enregistrée dans une variable :

$a = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]$.

Écrivez un code Dart qui prend cette liste et en crée une nouvelle qui ne contient que les éléments pairs de cette liste.



```
Tools Preview main.dart x exercice5.dart x Shell x exercice6.dart x exercice7.dart x
exercice7.dart
1 void main() {
2   final nombres = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100];
3   final nombresPairs = <int>[];
4
5   for (final nombre in nombres) {
6     if (nombre % 2 == 0) {
7       nombresPairs.add(nombre);
8     }
9   }
10
11   print('Nombres pairs : $nombresPairs');
12 }
```

```
~/workspace$ dart run exercice7.dart
Nombres pairs : [4, 16, 36, 64, 100]
~/workspace$
```

Exercice :8

Faites un jeu Pierre-Papier-Ciseaux à deux joueurs contre l'ordinateur.

```
Tools Preview main.dart x exercice5.dart x exercice8.dart x Shell x
exercice8.dart
1 import 'dart:io';
2 import 'dart:math';
3
4 void main() {
5   const choixPossibles = ['pierre', 'papier', 'ciseaux'];
6   final generateur = Random();
7
8   stdout.write('Choisissez pierre, papier ou ciseaux : ');
9   final choixJoueur = stdin.readLineSync()?.trim().toLowerCase();
10
11   if (choixJoueur == null || !choixPossibles.contains(choixJoueur)) {
12     print('Choix invalide. Écrivez pierre, papier ou ciseaux.');
```

```
Tools Preview main.dart x exercice5.dart x
~/workspace: dart run exercice8.dart
~/workspace$ dart run exercice8.dart
Choisissez pierre, papier ou ciseaux : ciseaux
Vous avez choisi : ciseaux
L'ordinateur a choisi : ciseaux
Égalité !
~/workspace$ dart run exercice8.dart
Choisissez pierre, papier ou ciseaux : pierre
Vous avez choisi : pierre
L'ordinateur a choisi : ciseaux
Vous avez gagné !
~/workspace$
```

Exercice :9

Générez un nombre aléatoire entre 1 et 100. Demandez à l'utilisateur de deviner le nombre, puis dites-lui s'il a deviné trop bas, trop haut ou exactement juste.

```

Tools Preview main.dart x exercice5.dart x exercice8.dart x She
exercice9.dart
1  import 'dart:io';
2  import 'dart:math';
3
4  void main() {
5      final nombreSecret = Random().nextInt(100) + 1;
6      var nombreTentatives = 0;
7      int? proposition;
8
9      print('J\'ai choisi un nombre entre 1 et 100.');
```

```

vous avez gagné !
~/workspace$ dart run exercice9.dart
J'ai choisi un nombre entre 1 et 100.
Devinez le nombre : 3
Trop bas !
Devinez le nombre : 50
Trop haut !
Devinez le nombre : 40
Trop haut !
Devinez le nombre : 20
Trop bas !
Devinez le nombre : 25
Trop bas !
Devinez le nombre : 28
Trop bas !
Devinez le nombre : 30
Trop bas !
Devinez le nombre : 42
Trop haut !
Devinez le nombre : 40
Trop haut !
Devinez le nombre : 39
Trop haut !
Devinez le nombre : 38
Trop haut !
Devinez le nombre : 33
Trop haut !
Devinez le nombre : 32
Bravo, vous avez trouvé !
Nombre de suppositions : 13
~/workspace$
```

Exercice :10

Demandez à l'utilisateur un nombre et déterminez si le nombre est premier ou non.

```
Tools  Preview  main.dart  x  Shell  x  exercice9.dart  x  exercice10.d...
exercice10.dart
1  import 'dart:io';
2
3  bool estPremier(int nombre) {
4    if (nombre < 2) {
5      return false;
6    }
7
8    for (var diviseur = 2; diviseur * diviseur <= nombre; diviseur++) {
9      if (nombre % diviseur == 0) {
10       return false;
11     }
12   }
13
14   return true;
15 }
16
17 void main() {
18   stdout.write('Entrez un nombre entier : ');
19   final texte = stdin.readLineSync()?.trim();
20   final nombre = int.tryParse(texte ?? '');
21
22   if (nombre == null) {
23     print('Veuillez entrer un nombre entier valide.');
```

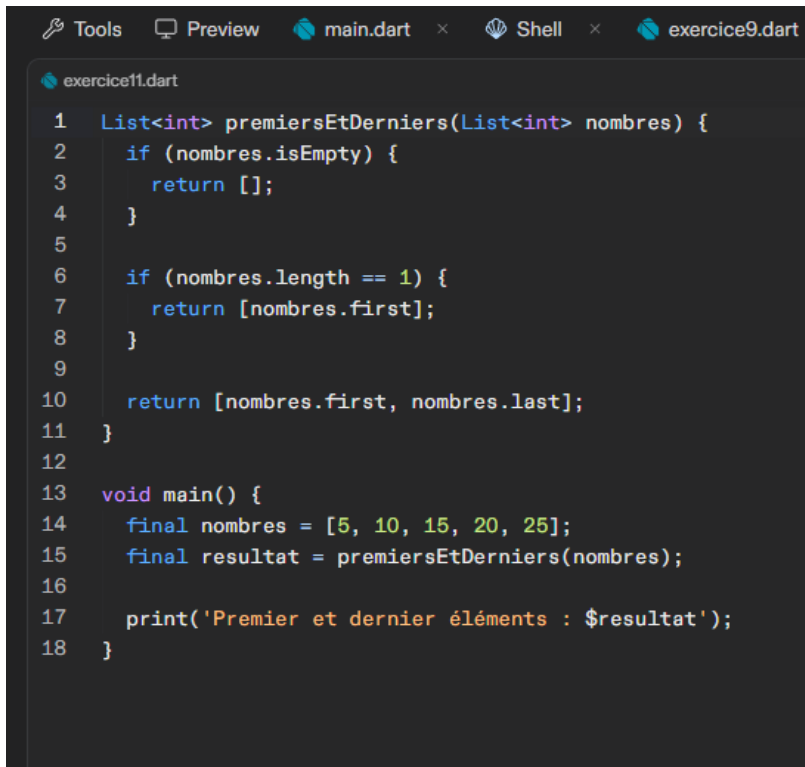
```
~/workspace$ dart run exercice10.dart
Entrez un nombre entier : 12
12 n'est pas un nombre premier.
~/workspace$ dart run exercice10.dart
17Entrez un nombre entier : 17
1717 n'est pas un nombre premier.
~/workspace$
```

Exercice :11

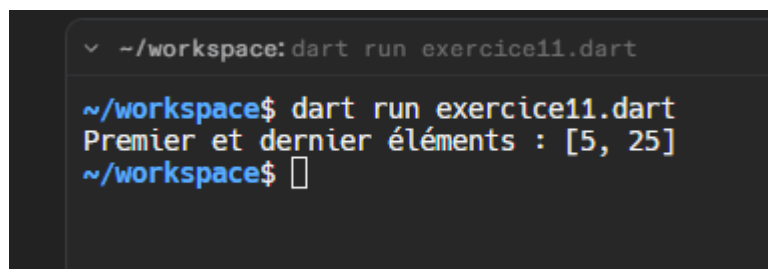
Écrire un programme qui prend une liste de nombres par exemple

$a = [5, 10, 15, 20, 25]$

et fait une nouvelle liste des seuls premiers et derniers éléments de la liste donnée. Pour vous entraîner, écrivez ce code dans une fonction.



```
exercice11.dart
1 List<int> premiersEtDerniers(List<int> nombres) {
2   if (nombres.isEmpty) {
3     return [];
4   }
5
6   if (nombres.length == 1) {
7     return [nombres.first];
8   }
9
10  return [nombres.first, nombres.last];
11 }
12
13 void main() {
14   final nombres = [5, 10, 15, 20, 25];
15   final resultat = premiersEtDerniers(nombres);
16
17   print('Premier et dernier éléments : $resultat');
18 }
```



```
~/workspace: dart run exercice11.dart
~/workspace$ dart run exercice11.dart
Premier et dernier éléments : [5, 25]
~/workspace$
```

Exercice :12

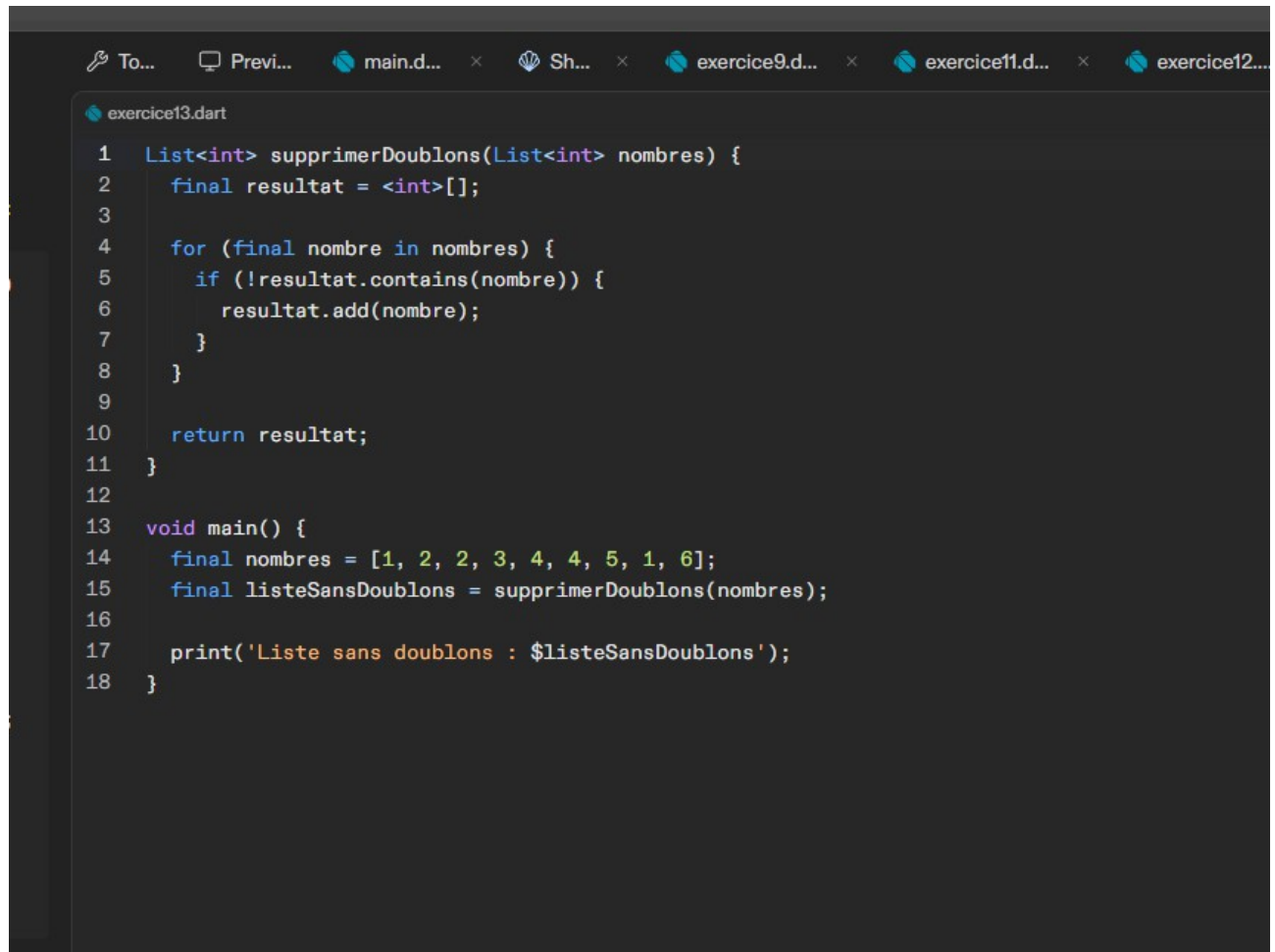
Écrivez un programme qui demande à l'utilisateur combien de nombres de Fibonacci il doit générer, puis les génère. Profitez de cette occasion pour réfléchir à la façon dont vous pouvez utiliser les fonctions.

```
Tools Preview main.dart x Shell x exercice9.dart x exercice11.dart x exercice12.dart
exercice12.dart
1 import 'dart:io';
2
3 List<int> genererFibonacci(int nombreTermes) {
4   final suite = <int>[];
5
6   if (nombreTermes <= 0) {
7     return suite;
8   }
9
10  suite.add(0);
11
12  if (nombreTermes == 1) {
13    return suite;
14  }
15
16  suite.add(1);
17
18  for (var i = 2; i < nombreTermes; i++) {
19    suite.add(suite[i - 1] + suite[i - 2]);
20  }
21
22  return suite;
23 }
24
25 void main() {
26   stdout.write('Combien de nombres de Fibonacci voulez-vous générer ? ');
27   final texte = stdin.readLineSync()?.trim();
28   final nombreTermes = int.tryParse(texte ?? '');
29
30   if (nombreTermes == null || nombreTermes < 0) {
31     print('Veuillez entrer un nombre entier positif ou nul.');
```

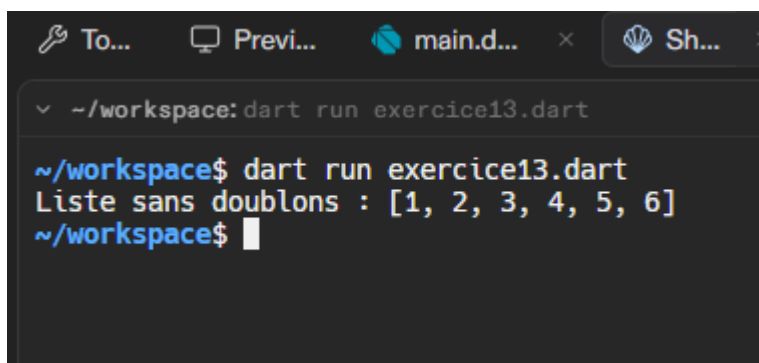
```
Tools Preview main.dart x Shell x exercice9.dart x exercice11.dart x exercice12.d... x +
~/workspace:dart run exercice12.dart
~/workspace$ dart run exercice11.dart
Premier et dernier éléments : [5, 25]
~/workspace$ dart run exercice12.dart
Combien de nombres de Fibonacci voulez-vous générer ? 21
Suite de Fibonacci : [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765]
~/workspace$
```

Exercice :13

Écrivez un programme (fonction) qui prend une liste et renvoie une nouvelle liste qui contient tous les éléments de la première liste moins tous les doublons.



```
exercice13.dart
1 List<int> supprimerDoublons(List<int> nombres) {
2   final resultat = <int>[];
3
4   for (final nombre in nombres) {
5     if (!resultat.contains(nombre)) {
6       resultat.add(nombre);
7     }
8   }
9
10  return resultat;
11 }
12
13 void main() {
14   final nombres = [1, 2, 2, 3, 4, 4, 5, 1, 6];
15   final listeSansDoublons = supprimerDoublons(nombres);
16
17   print('Liste sans doublons : $listeSansDoublons');
18 }
```



```
~/workspace: dart run exercice13.dart
~/workspace$ dart run exercice13.dart
Liste sans doublons : [1, 2, 3, 4, 5, 6]
~/workspace$
```

Exercice :14

Écrivez un programme (en utilisant des fonctions !) qui demande à l'utilisateur une longue chaîne contenant plusieurs mots. Renvoyez à l'utilisateur la même chaîne, sauf avec les mots dans l'ordre inverse.

```
exercice14.dart
1  import 'dart:io';
2
3  String inverserLesMots(String phrase) {
4      final phraseNettoyee = phrase.trim();
5
6      if (phraseNettoyee.isEmpty) {
7          return '';
8      }
9
10     final mots = phraseNettoyee.split(RegExp(r'\s+'));
11     return mots.reversed.join(' ');
12 }
13
14 void main() {
15     stdout.write('Entrez une phrase : ');
16     final phrase = stdin.readLineSync() ?? '';
17     final phraseInversee = inverserLesMots(phrase);
18
19     print('Phrase avec les mots inversés : $phraseInversee');
20 }
```

```
~/workspace: dart run exercice14.dart
~/workspace$ dart run exercice13.dart
Liste sans doublons : [1, 2, 3, 4, 5, 6]
~/workspace$ dart run exercice14.dart
Entrez une phrase : bonour les amis
Phrase avec les mots inversés : amis les bonour
~/workspace$
```

Exercice :15

Écrivez un générateur de mot de passe dans Dart. Soyez créatif avec la façon dont vous générez des mots de passe - les mots de passe forts contiennent un mélange de lettres minuscules, de lettres majuscules, de chiffres et de symboles. Les mots de passe doivent être aléatoires, générant un nouveau mot de passe chaque fois que l'utilisateur demande un nouveau mot de passe. Incluez votre code d'exécution dans une méthode principale.

```
import 'dart:io';
import 'dart:math';

final generateur = Random.secure();

const motsFaibles = [
  'soleil',
  'chat',
  'fleur',
  'orange',
  'banane',
  'montagne',
  'musique',
  'vacances',
];

const minuscules = 'abcdefghijklmnopqrstuvwxyz';
const majuscules = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
const chiffres = '0123456789';
const symboles = '!@#$%^&*()_-=+';

String choisirCaractere(String caracteres) {
  return caracteres[generateur.nextInt(caracteres.length)];
}

String melanger(List<String> caracteres) {
  for (var i = caracteres.length - 1; i > 0; i--) {
    final positionAleatoire = generateur.nextInt(i + 1);
    final temporaire = caracteres[i];
    caracteres[i] = caracteres[positionAleatoire];
    caracteres[positionAleatoire] = temporaire;
  }

  return caracteres.join();
}

String motDePasseFaible() {
  final premierMot = motsFaibles[generateur.nextInt(motsFaibles.length)];

  if (generateur.nextBool()) {
    return premierMot;
  }

  var deuxiemeMot = motsFaibles[generateur.nextInt(motsFaibles.length)];
  while (deuxiemeMot == premierMot) {
    deuxiemeMot = motsFaibles[generateur.nextInt(motsFaibles.length)];
  }

  return '$premierMot$deuxiemeMot';
}

String motDePasseMoyen() {
  const caracteres = '$minuscules$majuscules$chiffres';
  final motDePasse = <String>[];

  for (var i = 0; i < 10; i++) {
```

```
~/workspace: dart run exercice15.dart

~/workspace$ dart run exercice15.dart
Générateur de mots de passe
1 - Faible (un ou deux mots)
2 - Moyen (lettres et chiffres)
3 - Fort (lettres, chiffres et symboles)

Choisissez un niveau ou tapez "quitter" : 2
Mot de passe généré : 6yihQFc8l9
Ne l'utilisez pas pour un compte réel.

Choisissez un niveau ou tapez "quitter" : 
```

Exercice :16

Créez un programme qui jouera le jeu « des vaches et des taureaux » avec l'utilisateur. Le jeu fonctionne comme ceci :

- Générez aléatoirement un nombre à 4 chiffres. Demandez à l'utilisateur de deviner un nombre à 4 chiffres. Pour chaque chiffre que l'utilisateur a deviné correctement au bon endroit, il a une "vache". Pour chaque chiffre que l'utilisateur a deviné correctement au mauvais endroit est un "taureau".
- Chaque fois que l'utilisateur fait une supposition, dites-lui combien de "vaches" et de "taureaux" il a. Une fois que l'utilisateur a deviné le bon numéro, le jeu est terminé. Gardez une trace du nombre de suppositions que l'utilisateur fait tout au long du jeu et dites-le à l'utilisateur à la fin.

```
exercice16.dart
1 import 'dart:io';
2 import 'dart:math';
3
4 final generateur = Random.secure();
5
6 String genererNombreSecret() {
7   final chiffresDisponibles = List.generate(10, (index) => index);
8   final premierChiffre = generateur.nextInt(9) + 1;
9
10  chiffresDisponibles.remove(premierChiffre);
11  chiffresDisponibles.shuffle(generateur);
12
13  final chiffres = [
14    premierChiffre,
15    ...chiffresDisponibles.take(3),
16  ];
17
18  return chiffres.join();
19 }
20
21 int compterVaches(String secret, String proposition) {
22   var vaches = 0;
23
24   for (var i = 0; i < 4; i++) {
25     if (secret[i] == proposition[i]) {
26       vaches++;
27     }
28   }
29
30   return vaches;
31 }
32
33 int compterTaureaux(String secret, String proposition, int vaches) {
34   final occurrencesSecret = List.filled(10, 0);
35   final occurrencesProposition = List.filled(10, 0);
36
37   for (var i = 0; i < 4; i++) {
38     occurrencesSecret[int.parse(secret[i])]++;
39     occurrencesProposition[int.parse(proposition[i])]++;
40   }
41
42   AI () Dart 0 0 0 0 Diff
```

```
Au revoir !
~/workspace$ dart run exercice16.dart
J'ai choisi un nombre à 4 chiffres.
Une vache = chiffre bien placé.
Un taureau = chiffre présent mais mal placé.

Devinez le nombre (ou tapez "quitter") : 50
Entrez exactement 4 chiffres, sans commencer par 0.

Devinez le nombre (ou tapez "quitter") : 1
Entrez exactement 4 chiffres, sans commencer par 0.

Devinez le nombre (ou tapez "quitter") : 1234
Vaches : 0 | Taureaux : 1

Devinez le nombre (ou tapez "quitter") : 2134
Vaches : 0 | Taureaux : 1
```

Exercice :17

C'est l'heure des faux graphismes ! Disons que nous voulons dessiner des plateaux de jeu qui ressemblent à ceci :

```
exercice17.dart
1 import 'dart:io';
2
3 void dessinerPlateau(int taille) {
4   final ligneHorizontale = ' ${List.filled(taille, '---').join(' ')} ';
5   final ligneVerticale = '|${List.filled(taille, '  ').join('|')}|';
6
7   for (var ligne = 0; ligne < taille; ligne++) {
8     print(ligneHorizontale);
9     print(ligneVerticale);
10  }
11
12  print(ligneHorizontale);
13 }
14
15 void main() {
16   stdout.write('Quelle taille de plateau voulez-vous dessiner ? ');
17   final texte = stdin.readLineSync()?.trim();
18   final taille = int.tryParse(texte ?? '');
19
20   if (taille == null || taille <= 0) {
21     print('Veuillez entrer un nombre entier positif.');
```

```
~/workspace:dart run exercice17.dart
~/workspace$ dart run exercice17.dart
Quelle taille de plateau voulez-vous dessiner ? 4
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
~/workspace$
```

Exercice :18

Comme vous l'avez peut-être deviné, nous essayons de créer une planche complète de tic-tac-toe. Pour l'instant, nous nous concentrerons simplement sur la vérification si quelqu'un a GAGNÉ le jeu, sans nous soucier de la façon dont les mouvements ont été effectués.

```

1  int? trouverGagnant(List<List<int>> plateau) {
2    for (var ligne = 0; ligne < 3; ligne++) {
3      if (plateau[ligne][0] != 0 &&
4          plateau[ligne][0] == plateau[ligne][1] &&
5          plateau[ligne][1] == plateau[ligne][2]) {
6        return plateau[ligne][0];
7      }
8    }
9
10   for (var colonne = 0; colonne < 3; colonne++) {
11     if (plateau[0][colonne] != 0 &&
12         plateau[0][colonne] == plateau[1][colonne] &&
13         plateau[1][colonne] == plateau[2][colonne]) {
14       return plateau[0][colonne];
15     }
16   }
17
18   if (plateau[0][0] != 0 &&
19       plateau[0][0] == plateau[1][1] &&
20       plateau[1][1] == plateau[2][2]) {
21     return plateau[0][0];
22   }
23
24   if (plateau[0][2] != 0 &&
25       plateau[0][2] == plateau[1][1] &&
26       plateau[1][1] == plateau[2][0]) {
27     return plateau[0][2];
28   }
29
30   return null;
31 }
32
33 void main() {
34   final jeu = [
35     [1, 2, 0],
36     [2, 1, 0],
37     [2, 1, 1],
38   ];
39
40   final gagnant = trouverGagnant(jeu);
41

```

```

~/workspace$ dart run exercice18.dart
Le joueur 1 a gagné !
~/workspace$

```

Exercice :19

Dans un exercice précédent, nous avons exploré l'idée d'utiliser une liste de listes comme une "structure de données" pour stocker des informations sur un jeu de tic tac toe. Dans un jeu de tic tac toe, le « serveur de jeu » doit savoir où se trouvent les X et les O sur le plateau, pour savoir si le joueur 1 ou le joueur 2 (ou celui qui est X et O) a gagné.

```

Tools Preview main.dart x Shell x exercice16.d... x exercice17.dart x exercice19.d...
exercice19.dart
1  import 'dart:io';
2
3  String symboleDuJoueur(int joueur) {
4    if (joueur == 1) {
5      return 'X';
6    }
7
8    if (joueur == 2) {
9      return 'O';
10   }
11
12   return ' ';
13 }
14
15 void afficherPlateau(List<List<int>> plateau) {
16   print(' --- --- --- ');
17
18   for (final ligne in plateau) {
19     final cases = ligne.map(symboleDuJoueur).join(' | ');
20     print('| $cases |');
21     print(' --- --- --- ');
22   }
23 }
24
25 List<int>? demanderCoordonnees() {
26   while (true) {
27     stdout.write('Entrez la ligne et la colonne (exemple : 1,3) : ');
28     final saisie = stdin.readLineSync()?.trim() ?? '';
29
30     if (saisie.toLowerCase() == 'quitter') {
31       return null;
32     }
33
34     final coordonnees = saisie.split(',');
35
36     if (coordonnees.length != 2) {
37       print('Format invalide. Utilisez ligne,colonne, par exemple 1,3.');

```

```

Jeu de morpion pour deux joueurs
Le joueur 1 joue avec X et le joueur 2 avec O.
Tapez "quitter" pour arrêter la partie.

--- --- ---
| | | |
| | | |
| | | |
Joueur 1 (X)
Entrez la ligne et la colonne (exemple : 1,3) : 2,1

--- --- ---
| | | |
| X | | |
| | | |
Joueur 2 (O)
Entrez la ligne et la colonne (exemple : 1,3) : 3,1

--- --- ---
| | | |
| X | | |
| O | | |
Joueur 1 (X)

```

Exercice :20

```

1 import 'dart:io';
2
3 String symboleDuJoueur(int joueur) {
4   return joueur == 1 ? 'X' : 'O';
5 }
6
7 void afficherPlateau(List<List<int>> plateau) {
8   print(' --- --- ');
9
10  for (final ligne in plateau) {
11    final cases = ligne
12      .map((joueur) => joueur == 0 ? ' ' : symboleDuJoueur(joueur))
13      .join(' | ');
14    print('| $cases |');
15    print(' --- --- ');
16  }
17 }
18
19 int? trouverGagnant(List<List<int>> plateau) {
20   const lignesGagnantes = <List<List<int>>>[
21     [
22       [0, 0],
23       [0, 1],
24       [0, 2],
25     ],
26     [
27       [1, 0],
28       [1, 1],
29       [1, 2],
30     ],
31     [
32       [2, 0],
33       [2, 1],
34       [2, 2],
35     ],
36     [
37       [0, 0],
38       [1, 0],
39       [2, 0],
40     ],
41   ];
42 }

```

```

Joueur 2 (O)
Entrez la ligne et la colonne, par exemple 1,3 : 1,1

 --- --- ---
| 0 | X |  |
|  |  |  |
|  |  |  |
 --- --- ---
Joueur 1 (X)
Entrez la ligne et la colonne, par exemple 1,3 : 2,2

 --- --- ---
| 0 | X |  |
|  | X |  |
|  |  |  |
 --- --- ---
Joueur 2 (O)

```

ALOUANI Fadi

Exercice :21

Exercice :22

Exercice :23

Exercice :24

Exercice :25

Exercice :26

Conclusion

Ces exercices m'ont permis de mieux comprendre le langage Dart et de mettre en pratique ses principales fonctionnalités. Ils constituent une bonne base pour commencer ensuite à utiliser Flutter.